



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Assignment 1

Student Name: Ashutosh Yadav

UID: 23BCS11023

Branch: BE-CSE

Section/Group: KRG-3A

Semester: 6th

Date of Performance: 04/02/2026

Subject Name: System Design

Subject Code: 23CSH-314

Q1. Explain the role of interfaces and enums in software design with proper examples.

Interfaces

What they are: A contract that defines *what* operations a type must provide, without forcing *how* they're implemented.

Why they matter in design

- **Abstraction:** depend on behavior, not concrete classes.
- **Extensibility:** add new implementations without changing existing business logic.
- **Testability:** easily swap real implementations with mocks/fakes.
- **Separation of concerns:** business logic stays independent of infrastructure (DB, APIs, gateways).

Example (Java)

// Contract

```
interface Notifier {  
    void send(String userId, String message);  
}
```

// Implementations

```
class EmailNotifier implements Notifier {  
    public void send(String userId, String message) { /* send email */ }
```

```
}
```

```
class SmsNotifier implements Notifier {  
    public void send(String userId, String message) { /* send SMS */ }  
}
```

```
// Usage (depends on interface, not class)
```

```
class OrderService {  
    private final Notifier notifier;  
  
    OrderService(Notifier notifier) { this.notifier = notifier; }  
  
    void placeOrder(String userId) {  
        // order logic  
        notifier.send(userId, "Order placed successfully.");  
    }  
}
```

Here, OrderService doesn't care whether it's email/SMS/WhatsApp. That's good design.

Enums

What they are: A type-safe set of fixed named constants.

Why they matter in design

- **Prevents invalid values:** avoids strings like "paid", "Paid", "PAID" causing bugs.
- **Improves readability:** code becomes self-documenting.
- **Enables exhaustive handling:** compilers/IDEs warn if you forgot a case.
- **Centralizes domain states:** statuses and categories become consistent across services.

Example (Java)

```
enum OrderStatus {  
    CREATED, PAID, SHIPPED, DELIVERED, CANCELLED  
}
```

```
class Order {  
    private OrderStatus status = OrderStatus.CREATED;
```

```
void markPaid() { status = OrderStatus.PAID; }  
}
```

Interfaces + Enums together

- Use **interfaces** for “pluggable behavior”
- Use **enums** for “fixed domain states and categories”

Q2. Discuss how interfaces enable loose coupling with example.

Loose coupling means

A module should **not** be tightly dependent on another module’s concrete implementation. It should depend on an abstraction.

Why interfaces help

- If code depends on PaymentGateway (interface), you can replace RazorpayGateway with StripeGateway without rewriting business logic.
- Changes remain localized to the implementation class, not the whole system.

Tightly coupled (bad)

```
class CheckoutService {  
    private final RazorpayGateway gateway = new RazorpayGateway(); // fixed dependency  
  
    void pay(int amount) {  
        gateway.charge(amount); // cannot switch easily  
    }  
}
```

Loosely coupled (good)

```
interface PaymentGateway {  
    String charge(int amount);  
}  
  
class RazorpayGateway implements PaymentGateway {  
    public String charge(int amount) { return "razorpay_txn_123"; }  
}  
  
class StripeGateway implements PaymentGateway {  
    public String charge(int amount) { return "stripe_txn_abc"; }  
}
```

```
class CheckoutService {  
    private final PaymentGateway gateway;  
  
    CheckoutService(PaymentGateway gateway) { this.gateway = gateway; }  
  
    void pay(int amount) {  
        String txnId = gateway.charge(amount);  
        // continue workflow  
    }  
}
```

Result: CheckoutService stays the same even if you switch gateways, add mocks for testing, or introduce a new provider.